API MANUAL TESTING CHECKLIST

Backend Technical Test — PT Serasi Autoraya (SERA / Astra Group)

NestJS · TypeScript · PostgreSQL · Redis · Bull Queue

Candidate	**Bintang Wijaya**	**Tanggal Test**	___ / ___ / 2026
Email	bintangwijaya18@gmail.com	**Tester / QA**	_____
Position	Backend Developer	**Base URL**	https://_____

INFO Dokumen ini adalah checklist manual testing. Setiap baris dengan simbol ☐ dapat dicentang menggunakan pena setelah dicetak. Jalankan endpoint sesuai urutan yang disarankan. Centang ☐ PASS atau ☐ FAIL di bagian bawah setiap endpoint setelah selesai diuji.

Checklist Progress Summary

Tandai setelah seluruh endpoint dalam tiap grup selesai diuji:

✓	Group	Scope
☐	**0. Pre-Test Setup**	Health check, docker up, seeder berjalan, tools siap
☐	**1. Auth — Register**	POST /auth/register — semua skenario diuji
☐	**2. Auth — Login**	POST /auth/login — token tersimpan
☐	**3. Product — List**	GET /products — pagination & search
☐	**4. Product — Detail**	GET /products/:id
☐	**5. Product — Create**	POST /products — admin only
☐	**6. Product — Update**	PATCH /products/:id
☐	**7. Product — Delete**	DELETE /products/:id — soft delete verified
☐	**8. Order — Create**	POST /orders — stock deduction & queue verified
☐	**9. Order — History**	GET /orders — RBAC isolation verified
☐	**10. Order — Detail**	GET /orders/:id — price snapshot verified
☐	**11. System — Health**	GET /health — semua service connected

✓	Group	Scope
☐	**12. System — Metrics**	GET /metrics — counters verified
☐	**13. Queue Verification**	Email terkirim, activity log tersimpan, retry/DLQ verified
☐	**14. Security Test**	Rate limit, JWT expire, RBAC escalation, SQL injection

0. Pre-Test Setup Checklist

✓	Pre-Test Item
☐	docker ps → semua container (app, postgres, redis) berstatus Up
☐	GET /health → response: { status: "ok", database: "connected", redis: "connected" }
☐	Tabel users, products, orders, order_items, activity_logs sudah ada di database
☐	Seeder berjalan → admin@sera.test / Admin123! dan customer@sera.test / Customer123! bisa login
☐	File .env sudah terisi lengkap (DB, JWT, SMTP, REDIS)
☐	Tool testing siap (Postman / Insomnia / curl)
☐	Catat BASE_URL yang akan digunakan: http://_____:3000

```
# Default Credentials (Seeder)
Admin    : admin@sera.test      / Admin123!
Customer : customer@sera.test   / Customer123!

# Simpan token setelah login:
ADMIN_TOKEN    = eyJ... (dari response POST /auth/login dengan akun admin)
CUSTOMER_TOKEN = eyJ... (dari response POST /auth/login dengan akun customer)
```

1. Authentication Endpoints

INFO Jalankan Register dulu untuk buat akun baru, lalu Login untuk mendapatkan token. Token admin dan customer disimpan terpisah.

POST /auth/register — Register User

Auth Required	✗ No — Public endpoint	**Role Access**	Public
Description	Mendaftarkan user baru. Password di-hash bcrypt. Default role: customer.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Content-Type	application/json	REQUIRED	Wajib

Request Body (Content-Type: application/json)

Field	Type	Required?	Deskripsi & Contoh Nilai
name	string	REQUIRED	Nama lengkap, min 2 karakter. Contoh: "Bintang Wijaya"
email	string	REQUIRED	Email valid & unik. Contoh: "bintang@autobot.co.id"
password	string	REQUIRED	Min 8 karakter, kombinasi huruf & angka. Contoh: "P@ssword123"
role	string	Optional	Nilai: "admin" atau "customer". Default: "customer"

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	201	Created	Register berhasil	*{ success:true, data:{ user:{...}, access_token:"eyJ..." } }*
☐	400	Bad Request	Field kosong / format salah	*{ success:false, errors:[{ field:"email", message:"..." }] }*
☐	409	Conflict	Email sudah terdaftar	*{ success:false, message:"Email already registered" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Register dengan email yang sudah ada → harus return 409 Conflict.
☐	Register tanpa field name / email / password → harus return 400 dengan detail errors per field.
☐	Password kurang dari 8 karakter → harus return 400.
☐	Format email tidak valid (mis. "bintang@") → harus return 400.
☐	Pastikan field "password" TIDAK muncul di response body sama sekali.

☐	Token di response langsung valid untuk dipakai hit endpoint yang butuh auth.
☐	Register dengan role "admin" → pastikan role tersimpan benar di database.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

POST /auth/login — Login User

Auth Required	✗ No — Public endpoint	**Role Access**	Public
Description	Login dengan email & password. Mengembalikan JWT Bearer Token (expires 1 jam).		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Content-Type	application/json	REQUIRED	Wajib

Request Body (Content-Type: application/json)

Field	Type	Required?	Deskripsi & Contoh Nilai
email	string	REQUIRED	Email terdaftar. Contoh: "bintang@autobot.co.id"
password	string	REQUIRED	Password akun. Contoh: "P@ssword123"

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Login berhasil	*{ success:true, data:{ access_token:"eyJ...", expires_in:3600 } }*
☐	400	Bad Request	Field tidak lengkap	*{ success:false, errors:[...] }*
☐	401	Unauthorized	Email atau password salah	*{ success:false, message:"Invalid credentials" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Simpan access_token dari response — wajib digunakan di semua endpoint yang butuh auth.
☐	Login dengan password salah → harus return 401, BUKAN 400.
☐	Login dengan email yang belum terdaftar → harus return 401.
☐	Decode token di jwt.io — pastikan payload berisi: sub (user id), email, role.
☐	Tunggu 1+ jam, gunakan token lama → harus return 401 Unauthorized (token expired).
☐	Login dengan body kosong → harus return 400.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

2. Product Endpoints

NOTE RBAC: GET (public), POST/PATCH/DELETE (admin only). Selalu uji akses dengan customer token untuk verifikasi 403.

GET /products — List Products — Pagination & Search

Auth Required	✗ No — Public endpoint	**Role Access**	Public
Description	List semua produk aktif dengan pagination. Mendukung search by name dan sort.		

Query Parameters

Parameter	Type	Required?	Default / Contoh
page	integer	Optional	1 (default: 1)
limit	integer	Optional	10 (default: 10, max: 100)
search	string	Optional	laptop
sort	string	Optional	price | name | created_at
order	string	Optional	asc | desc (default: desc)

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Data ditemukan	*{ data:[...], meta:{ page:1, limit:10, total:50, totalPages:5 } }*
☐	200	OK	Search tidak ada hasil	*{ data:[], meta:{ total:0, totalPages:0 } }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Hit tanpa params → default page=1, limit=10.
☐	Hit ?page=2&limit=5 → data harus berbeda dari page=1.
☐	Hit ?search=namaproduk → hanya produk yang mengandung kata tersebut yang muncul.
☐	Hit ?search=zzznoproduct → harus return data:[] (array kosong), BUKAN 404.
☐	Hit ?sort=price&order=asc → produk urut dari harga terendah ke tertinggi.
☐	Produk yang sudah soft-deleted TIDAK boleh muncul di listing.
☐	Hit ?limit=200 → pastikan ada batasan maksimum atau handled gracefully.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

GET /products/:id — Get Product Detail

Auth Required	✗ No — Public endpoint	**Role Access**	Public
Description	Detail satu produk berdasarkan UUID.		

Query Parameters

Parameter	Type	Required?	Default / Contoh
:id	UUID	REQUIRED	a1b2c3d4-e5f6-7890-abcd-ef1234567890

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Produk ditemukan	*{ data:{ id, name, description, price, stock, created_at } }*
☐	400	Bad Request	Format ID bukan UUID valid	*{ message:"Validation failed" }*
☐	404	Not Found	ID tidak ada / sudah soft-deleted	*{ message:"Product not found" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Gunakan ID dari produk yang baru dibuat di POST /products.
☐	Hit dengan UUID random yang tidak ada di DB → harus return 404.
☐	Hit dengan format bukan UUID (mis. /products/abc atau /products/123) → harus return 400.
☐	Hit ID produk yang sudah di-DELETE (soft delete) → harus return 404.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

POST /products — Create Product

Auth Required	✓ Yes — Bearer JWT Token	**Role Access**	Admin Only
Description	Buat produk baru. Hanya admin yang bisa akses.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Content-Type	application/json	REQUIRED	-
Authorization	Bearer {admin_jwt}	REQUIRED	Token JWT dengan role=admin

Request Body (Content-Type: application/json)

Field	Type	Required?	Deskripsi & Contoh Nilai
name	string	REQUIRED	Min 2 karakter. Contoh: "Laptop ASUS ROG G15"

description	string	Optional	Deskripsi produk. Contoh: "Gaming laptop RTX 4070"
price	number	REQUIRED	Harga dalam Rupiah, harus > 0. Contoh: 25000000
stock	integer	REQUIRED	Jumlah stok tersedia, min 0. Contoh: 50

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	201	Created	Produk berhasil dibuat	*{ data:{ id, name, price, stock, created_at } }*
☐	400	Bad Request	Validasi field gagal	*{ errors:[{ field:"price", message:"..." }] }*
☐	401	Unauthorized	Tanpa token / token invalid	*{ message:"Unauthorized" }*
☐	403	Forbidden	Token role customer mencoba akses	*{ message:"Forbidden resource" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Hit tanpa Authorization header → harus return 401.
☐	Hit dengan token customer (bukan admin) → harus return 403.
☐	Hit dengan price = 0 atau negatif → harus return 400.
☐	Hit dengan stock = -1 → harus return 400.
☐	Hit dengan name kosong → harus return 400 dengan pesan error yang jelas.
☐	Catat ID produk yang berhasil dibuat — untuk test Update, Delete, dan Create Order.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

PATCH `/products/:id` — Update Product (Partial)

Auth Required	✓ Yes — Bearer JWT Token	**Role Access**	Admin Only
Description	Update sebagian field produk. Tidak perlu kirim semua field.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Content-Type	application/json	REQUIRED	-
Authorization	Bearer {admin_jwt}	REQUIRED	-

Request Body (Content-Type: application/json)

Field	Type	Required?	Deskripsi & Contoh Nilai
name	string	Optional	Update nama produk

description	string	Optional	Update deskripsi
price	number	Optional	Update harga
stock	integer	Optional	Update stok

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Update berhasil	*{ data:{ ...updated_product } }*
☐	400	Bad Request	Validasi field gagal	*{ errors:[...] }*
☐	401	Unauthorized	Tanpa token	*{ message:"Unauthorized" }*
☐	403	Forbidden	Role customer	*{ message:"Forbidden resource" }*
☐	404	Not Found	ID tidak ada	*{ message:"Product not found" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Kirim hanya { price: 30000000 } → hanya price yang berubah, field lain tetap sama.
☐	Setelah update, hit GET /products/:id → verifikasi nilai baru tersimpan.
☐	Hit update pada ID yang sudah di-soft-delete → harus return 404.
☐	Hit dengan customer token → harus return 403.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

DELETE `/products/:id` — Delete Product (Soft Delete)

Auth Required	**✓ Yes — Bearer JWT Token**	**Role Access**	Admin Only
Description	Soft delete — data tidak benar-benar dihapus, hanya set deleted_at timestamp.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Authorization	Bearer {admin_jwt}	REQUIRED	-

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Soft delete berhasil	*{ message:"Product deleted successfully" }*
☐	401	Unauthorized	Tanpa token	*{ message:"Unauthorized" }*
☐	403	Forbidden	Role customer	*{ message:"Forbidden resource" }*

☐	**404**	**Not Found**	ID tidak ada atau sudah hapus	*{ message:"Product not found" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan	
✓	**Test Case / Edge Case yang Harus Diverifikasi**
☐	Setelah DELETE, hit GET /products/:id dengan ID sama → harus return 404.
☐	Setelah DELETE, hit GET /products → produk tidak boleh muncul di list.
☐	DELETE dua kali pada ID yang sama → yang kedua harus return 404.
☐	Cek langsung di database: record masih ada dengan deleted_at terisi (bukan benar-benar dihapus).

HASIL ENDPOINT INI:	☐ **PASS**	☐ **FAIL**

3. Order Endpoints

NOTE Create Order menggunakan DB Transaction. Stock berkurang atomik. Email invoice & activity log diproses async via Bull Queue Redis.

POST /orders — Create Order

Auth Required	✓ Yes — Bearer JWT Token	**Role Access**	Customer Only
Description	Buat order baru. Stock otomatis dikurangi dalam DB transaction. Email invoice & activity log dikirim via Bull Queue secara async.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Content-Type	application/json	REQUIRED	-
Authorization	Bearer {customer_jwt}	REQUIRED	Token role customer
Idempotency-Key	uuid-unik-per-request	Optional	Prevent duplicate order jika request dikirim ulang

Request Body (Content-Type: application/json)

Field	Type	Required?	Deskripsi & Contoh Nilai
items	array	REQUIRED	Array minimal 1 item order
items[].product_id	UUID	REQUIRED	ID produk. Contoh: "a1b2c3d4-..."
items[].quantity	integer	REQUIRED	Jumlah yang dipesan, min 1

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	201	Created	Order berhasil dibuat	*{ data:{ id, total_price, status:"pending", items:[...] } }*
☐	400	Bad Request	Stok tidak cukup / validasi gagal	*{ message:"Insufficient stock for product X" }*
☐	401	Unauthorized	Tanpa token	*{ message:"Unauthorized" }*
☐	403	Forbidden	Admin token mencoba buat order	*{ message:"Forbidden resource" }*
☐	404	Not Found	product_id tidak ditemukan	*{ message:"Product not found" }*
☐	409	Conflict	Idempotency-Key duplikat	*{ message:"Duplicate order request" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Catat stok produk SEBELUM order → setelah order sukses stok harus berkurang sesuai quantity.
☐	Order dengan quantity melebihi stok yang tersedia → harus return 400.

☐	Order dengan product_id yang tidak ada → harus return 404.
☐	Kirim request yang sama 2x dengan Idempotency-Key yang sama → yang ke-2 harus return 409.
☐	Cek email invoice terkirim ke inbox email customer (via SMTP Hostinger).
☐	Cek tabel activity_logs di DB → harus ada row baru dengan action "order.created" dan entity_id = order_id.
☐	Hit dengan admin token → harus return 403.
☐	Order items kosong atau array kosong → harus return 400.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

GET /orders — Order History

Auth Required	✓ Yes — Bearer JWT Token	**Role Access**	Customer (milik sendiri) / Admin (semua)
Description	Customer hanya melihat history order miliknya sendiri. Admin melihat semua order.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Authorization	Bearer {token}	REQUIRED	Customer atau Admin JWT

Query Parameters

Parameter	Type	Required?	Default / Contoh
page	integer	Optional	1
limit	integer	Optional	10
status	string	Optional	pending | paid | cancelled

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Data berhasil diambil	*{ data:[...orders], meta:{ page, total, totalPages } }*
☐	401	Unauthorized	Tanpa token	*{ message:"Unauthorized" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Login sebagai Customer A → buat order → login sebagai Customer B → GET /orders: order A tidak boleh muncul.
☐	Login sebagai Admin → GET /orders: harus bisa melihat order dari semua user.
☐	Hit ?status=pending → hanya order berstatus pending yang muncul.

☐	Pastikan response menyertakan order_items (items detail) bukan hanya order header.

HASIL ENDPOINT INI:	□ PASS	□ FAIL

GET /orders/:id — Order Detail

Auth Required	✓ Yes — Bearer JWT Token	**Role Access**	Customer (milik sendiri) / Admin (semua)
Description	Detail satu order beserta semua item di dalamnya.		

Request Headers

Header Key	Value / Format	Required?	Keterangan
Authorization	Bearer {token}	REQUIRED	-

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Order ditemukan	*{ data:{ id, user_id, total_price, status, items:[...] } }*
☐	403	Forbidden	Customer akses order milik user lain	*{ message:"Forbidden resource" }*
☐	404	Not Found	Order ID tidak ditemukan	*{ message:"Order not found" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Akses order ID milik Customer A menggunakan token Customer B → harus return 403.
☐	Verifikasi harga di order_items adalah snapshot harga SAAT order (bukan harga current).
☐	Verifikasi total_price = SUM(quantity × price) dari semua items.
☐	Hit dengan UUID order yang tidak ada → harus return 404.

HASIL ENDPOINT INI:	□ PASS	□ FAIL

4. System Endpoints

GET /health — Health Check

Auth Required	✗ No — Public endpoint	**Role Access**	Public
Description	Cek status aplikasi: koneksi database PostgreSQL dan Redis.		

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Semua service up	*{ status:"ok", database:"connected", redis:"connected", uptime:3600 }*
☐	503	Service Unavailable	Salah satu service down	*{ status:"error", database:"disconnected" }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Hit endpoint ini PERTAMA sebelum mulai testing apapun.
☐	Tidak butuh auth — harus accessible tanpa token.
☐	Jika database atau redis "disconnected", stop dulu dan cek docker-compose logs.

HASIL ENDPOINT INI:	☐ PASS	☐ FAIL

GET /metrics — Application Metrics

Auth Required	✗ No — Public endpoint	**Role Access**	Public / Internal
Description	Metrics sederhana: total request, error, response time, dan status queue.		

Expected Responses — ☐ Centang jika response sesuai

✓	Code	Status	Kondisi / Trigger	Preview Response
☐	200	OK	Metrics berhasil diambil	*{ total_requests:142, total_errors:3, avg_response_time_ms:87, queue_pending:0, queue_failed:0 }*

Testing Notes & Edge Cases — ☐ Centang setelah dijalankan

✓	Test Case / Edge Case yang Harus Diverifikasi
☐	Cek queue_failed setelah semua test order → harus 0 jika queue berjalan normal.
☐	Cek total_requests bertambah setiap ada request masuk.
☐	Cek total_errors bertambah setelah sengaja hit endpoint yang error.

HASIL ENDPOINT INI:	□ PASS	□ FAIL

5. Async Queue Verification

NOTE Lakukan verifikasi berikut SETELAH minimal 1 order berhasil dibuat. Cek email, database, dan log aplikasi.

✓	Job / Fitur	Cara Verifikasi
☐	**SendInvoiceEmailJob**	Cek inbox email customer → email invoice diterima dengan detail order (ID, items, total harga)
☐	**SaveActivityLogJob**	Query DB: SELECT * FROM activity_logs WHERE entity_id = '{order_id}' → row baru dengan action "order.created" ada
☐	**SendWhatsAppNotificationJob**	Cek log app → baris "[WA Notification] Sent for order {id}" muncul (placeholder log)
☐	**Retry Mechanism**	Matikan SMTP sementara → buat order → hidupkan SMTP → email akhirnya terkirim setelah retry (max 3x, exponential backoff)
☐	**Dead Letter Queue**	Cek tabel failed_jobs setelah job gagal 3x → record tersimpan dengan error message lengkap
☐	**Idempotency (Email)**	Trigger job dengan order_id yang sama dua kali → email hanya terkirim 1x (cek Redis key "job:email:{orderId}")
☐	**Idempotency (Activity Log)**	Trigger job dengan order_id yang sama dua kali → hanya 1 row di activity_logs per order
☐	**Queue Empty After Test**	GET /metrics → queue_pending: 0 dan queue_failed: 0 setelah semua job selesai

6. Security & Edge Case Testing

✓	Test Case	Input / Cara Test	Expected Result
☐	**Rate Limiting**	Kirim 61+ request dalam 1 menit ke endpoint apapun dari IP yang sama	**429 Too Many Requests pada request ke-61**
☐	**SQL Injection**	Field email: ' OR 1=1; -- di POST /auth/login	**400 Bad Request — input ditolak validasi**
☐	**JWT Expired**	Gunakan token yang sudah expired (lebih dari 1 jam)	**401 Unauthorized — token expired**
☐	**JWT Malformed**	Authorization: Bearer invalid.token.here	**401 Unauthorized — invalid signature**
☐	**No Token**	Hit endpoint protected tanpa Authorization header	**401 Unauthorized**
☐	**Empty Bearer**	Authorization: Bearer (string kosong setelah Bearer)	**401 Unauthorized**
☐	**XSS Attempt**	name: "<script>alert(1)</script>" di POST /auth/register	**Input di-sanitize atau return 400**
☐	**CORS Block**	Hit dari origin yang tidak whitelist (gunakan browser atau curl --origin)	**403 atau blocked oleh CORS policy**
☐	**Secure Headers**	Cek response headers dari endpoint apapun	**Ada: X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security**
☐	**Password Exposure**	Lihat response body dari register, login, dan get user	**Field "password" tidak muncul di response body**
☐	**RBAC — Customer→Admin**	Gunakan customer token untuk POST /products, PATCH /products/:id, DELETE	**403 Forbidden untuk semua operasi**
☐	**RBAC — Admin→Order**	Gunakan admin token untuk POST /orders	**403 Forbidden — admin tidak bisa buat order**

7. Final Test Sign-Off

Endpoint / Group	☐ PASS	☐ FAIL
Auth — Register & Login	☐ PASS	☐ FAIL
Product — List & Detail (Public)	☐ PASS	☐ FAIL
Product — Create / Update / Delete (Admin)	☐ PASS	☐ FAIL
Order — Create (+ stock deduction)	☐ PASS	☐ FAIL
Order — History & Detail (RBAC)	☐ PASS	☐ FAIL
System — Health Check & Metrics	☐ PASS	☐ FAIL
Async Queue — Email, Log, Notification	☐ PASS	☐ FAIL
Security — Rate Limit, JWT, RBAC, Headers	☐ PASS	☐ FAIL

Tested by:	Date Completed:
Tanda Tangan & Nama	*Tanggal / Waktu Selesai*

8. HTTP Status Code Quick Reference

Code	Status	Digunakan Saat
200	OK	GET berhasil, PATCH berhasil, DELETE berhasil
201	Created	POST register / login / create product / create order berhasil
400	Bad Request	Validasi input gagal, format salah, stok tidak cukup, field missing
401	Unauthorized	Token tidak ada, invalid, atau sudah expired
403	Forbidden	Token valid tapi role tidak punya hak akses ke resource tersebut
404	Not Found	Resource dengan ID yang diberikan tidak ditemukan atau sudah dihapus
409	Conflict	Duplikasi: email sudah terdaftar, idempotency key duplikat
429	Too Many Requests	Rate limit terlampaui (60 request/menit per IP)
500	Internal Server Error	Error tak terduga di server — cek application log

Code	Status	Digunakan Saat
503	**Service Unavailable**	Database atau Redis sedang down